

Design and Implementation of a Composable Multi-Input UI Framework for 2D Games: A Case Study of Elysia Engine

Verified revision	dfac1cee3d90e2476140701cd9c432a8e4e0ca36
Verification date	July 20, 2026 (Pacific Daylight Time)
Implementation context	Elysia Engine engine/ui, hosted in the Moonline game repository

Abstract

Game user interfaces must work with several input devices, support nested menus, and keep focus correct while controls appear and disappear. These needs are difficult to handle if each widget implements its own routing rules. This paper presents the retained UI framework used by Elysia Engine, a C++23 2D game engine subsystem developed inside the Moonline project. The framework separates persistent UI elements, interactive controls, child-owning containers, local focus scopes, and window-level policies. `UiWindow` gives one place for focus-scope navigation, modal overlays, transient popups, passive tooltips, and scroll-target selection. Containers use `std::unique_ptr` for child ownership, while public raw pointers are normally borrowed views. A detailed `UiTabContainer` case study shows how composition, focus delegation, and the invariant `tab_count == page_count` work together. The paper also examines layout, styling, text, presentation effects, and render-command generation. Automated behavior tests and an interactive integration scene provide evidence for the described focus, ownership, routing, and composition behavior at the verified revision. The current implementation remains limited in cross-platform verification, accessibility support, and large-scale performance evaluation.

Keywords: game user interface, retained-mode UI, focus navigation, multi-input, composition, C++ ownership, Elysia Engine

1. Introduction

1.1 Problem background

A game UI is more than a set of buttons drawn on top of a scene. A main menu may be simple, but a settings screen can contain tab pages, scrollable sections, text fields, popups, tooltips, and a confirmation dialog at the same time. The same screen may be controlled by a mouse, keyboard, or gamepad. Each device has different expectations. A mouse points directly at a position. A keyboard or gamepad usually moves a logical focus. Text input needs character and composition events. An analog stick may also act as a repeated navigation or scrolling source.

SDL places keyboard, mouse, wheel, text, window, and controller data in the common `SDL_Event` union [1]. Its controller mapping support also helps normalize physical gamepads into stable logical gamepad controls [2]. Those services are a useful platform base, but they do not decide what a game UI action means. The engine still needs rules for actions such as Navigate Left, Confirm, Cancel, Home, or Page Down. It also needs to decide which control receives an action when a modal dialog, dropdown popup, or nested focus region is active.

Focus adds another source of complexity. The W3C keyboard-interface guidance separates movement between components from directional movement inside composite components [3]. Its Tabs Pattern also allows focus movement and tab activation to be separate operations [4]. These ideas come from web accessibility work, but the interaction distinction is also useful in a game menu. A player may move across tab labels without changing the page until Confirm is pressed. A focus target can also become invalid when a page is hidden or a child is removed. The framework must repair that state without leaving two controls focused or sending input to a destroyed object.

The UI programming model also affects the design. Dear ImGui describes a simple contrast between immediate-mode APIs and traditional toolkits that retain a widget tree [5]. Elysia Engine uses persistent C++ objects. This approach makes hierarchical layout, cached state, clipping, focus scopes, and long-lived composite controls natural. It also creates responsibilities: the tree needs explicit ownership, invalidation, cleanup, and state synchronization. This paper studies how the current framework handles those responsibilities. It does not argue that retained mode is always better than immediate mode.

1.2 Research question and scope

The main question is:

How can a small 2D game engine organize a persistent UI framework so that layout, ownership, multi-device input, nested focus, and layered surfaces remain composable and testable?

The paper answers this question through the current `elysia::ui` implementation. The scope includes the following areas:

- the persistent UI object model;
- child ownership and dynamic tree mutation;
- anchor, list, grid, and scroll layout;
- styles, themes, text, and presentation effects;
- normalization of keyboard, mouse, and gamepad input;
- local focus graphs and delegated nested focus;
- overlay, popup, tooltip, and scroll arbitration at the window level;
- composite components, with `UiTabContainer` as the main case study;
- automated and interactive verification available in the repository.

Elysia UI emits `UiRenderCommand` values; SDL execution of those commands belongs to the engine rendering layer. Moonline is the game project that hosts and uses the engine code. Throughout this paper, the engine is called **Elysia Engine**.

1.3 Main engineering contributions

The framework makes four main engineering choices.

First, it separates basic element state from interactive and child-owning capabilities. `UiControl` and `UiChildHost` form separate branches derived from `UiElement`; a container is not forced to pretend that it is one atomic button-like control.

Second, it makes child ownership explicit. Containers receive `std::unique_ptr<UiElement>` and return raw pointers only for non-owning access. This follows the general C++ guidance that smart pointers should express ownership transfer, while raw pointers normally represent non-owning access [6]. Stable child-lifetime records protect cached traversal handles when callbacks change the UI tree.

Third, it splits local mechanism from global policy. Focus scopes manage control-to-control navigation inside one region. `UiWindow` manages movement between scopes and decides whether a modal overlay, transient popup, tooltip, focused scroll area, or ordinary child tree has priority.

Fourth, it builds larger controls through composition. `UiTabContainer` owns a tab bar and a tab view, and `UiConfirmationDialog` owns a chrome container and action controls. The outer composite coordinates invariants, callbacks, delegated focus, and theme behavior instead of inheriting from a visually similar concrete widget.

These are not new algorithms. Their value lies in how they are combined into a coherent implementation with clear architectural boundaries.

2. Requirements and Design Goals

2.1 Functional requirements

The framework was developed for menus and game-facing screens in a 2D engine. Its main functional requirements are listed in Table 1.

Requirement	Framework response	Main owner
Persistent visual elements	Rectangle, order, opacity, visibility, presentation translation, and render-command submission	<code>UiElement</code>
Interactive controls	Enabled and focused state plus discrete input reception	<code>UiControl</code> and <code>UiFocusable</code>
Nested ownership	Exclusive child ownership, extraction, cleanup, update, input, and render traversal	<code>UiChildHost</code>
Reusable layout	Anchor, one-dimensional list, two-dimensional grid, padding, alignment, and scroll viewport	Layout helpers and containers
Multiple input devices	Device-independent UI actions plus pointer, text, wheel, and axis payloads	<code>UiInputRouter</code>
Directional navigation	Local control graph, neighboring scopes, nested focus delegation, and repair	Focus subsystem and <code>UiWindow</code>
Layered surfaces	Modal overlays, transient popups, passive tooltips, placement, dismissal, and focus restoration	<code>UiWindow</code>
Consistent presentation	Theme colors, field-level style overrides, typography roles, clipping, opacity, and translation effects	Style, text, effects, and render commands
Reusable complex controls	Tabs, dropdowns, dialogs, labeled controls, and settings preset	Composite layer

Table 1. Design requirements and their main owners.

2.2 Design goals

The framework is organized around **clear responsibility**. Input translation does not need to know how a tab page is built, a tooltip does not determine global z-order, and a layout helper does not own scene objects. Platform input, UI actions, local focus, window routing, and rendering therefore remain separate stages.

This separation also supports **composition**. A list can be nested inside a scroll container, placed within a tab page, and then hosted by a window without requiring leaf controls to understand every possible parent type.

Composition, however, also requires a **predictable lifetime model**. Child ownership should be visible in the type system, while traversal must remain safe when callbacks remove or reorder elements. Borrowed registrations are therefore pruned or explicitly unregistered when possible.

Input handling follows a related principle: UI actions are device-independent without discarding device context. A keyboard arrow and a gamepad direction can both become `UiAction::NavigateDown`, while the resulting event still records its `InputDevice`. This distinction matters because pointer-driven focus and gamepad-driven focus do not always follow the same policy.

Finally, important UI behavior should be observable as state transitions rather than only through a complete gameplay scene. Focus repair, popup cleanup, selection synchronization, style cascading, clipping, and callback behavior can therefore be verified independently, with the interactive UI scene serving as an integration surface.

2.3 Non-goals

The system is designed for game-facing UI rather than as a general desktop GUI toolkit. Features such as accessibility semantics, declarative UI, automatic data binding, touch-first input, and a full design-token system are outside its current scope.

3. Overall Architecture

3.1 Layered structure

The implementation is divided by responsibility rather than by screen. Figure 1 gives a simplified dependency view.

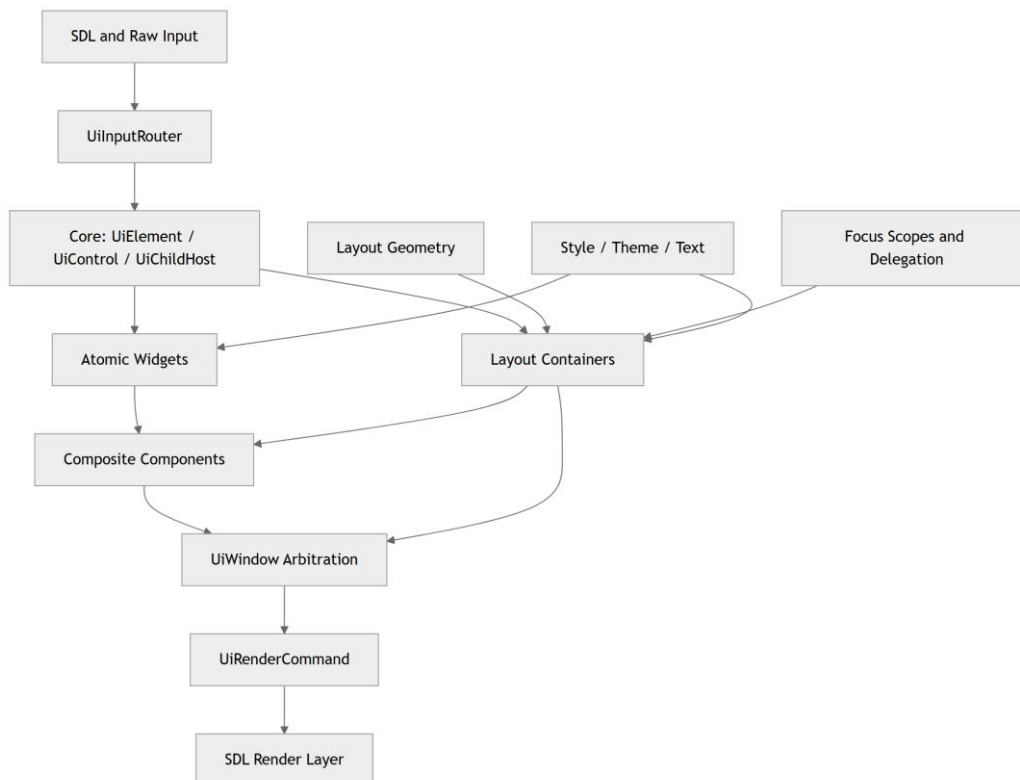


Figure 1. Main layers of the Elysia Engine UI framework.

`core` defines the base tree roles. `layout` contains reusable geometry rules. `widgets` contains atomic visual or interactive elements such as buttons, sliders, text inputs, labels, images, and bars. `containers` owns and arranges subtrees. `focus` describes local navigation regions and delegation. `composites` coordinates multiple widgets or containers into one useful component. `window` owns global interaction policies. `style`, `text`, and `effects` provide cross-cutting presentation data.

This is a retained object model. UI elements persist across frames and are updated when state changes. The system can therefore keep a focus graph, selected indices, scroll offsets, text-editing state, and presentation animations. The cost is that invalidation and cleanup must be correct when the tree changes.

3.2 The three core roles

`UiElement` is the common node. It derives from the engine's `SceneObject`, but it adds UI-specific screen-space geometry, visual order, opacity, presentation translation, and render-command submission. It also stores a non-owning layout-parent pointer. `screen_rect` stores the allocated layout geometry. Pointer hit testing normally uses `presentation_screen_rect()`, which adds the accumulated presentation translation. An entrance animation can therefore move the rendered and hit-tested result without rewriting the layout allocation every frame.

`UiControl` adds `UiFocusable`. `UiFocusable` is an input-receiver capability, and `UiControl` combines it with an element. This branch represents atomic interaction. A button, checkbox, slider, or text field can be enabled, focused, and asked to consume an event.

`UiChildHost` takes the other branch. It combines `UiElement` with update, frame-input, and discrete-event receivers. It owns a vector of `ChildEntry` records. Each entry contains a `std::unique_ptr<UiElement>`, layout options, style-relation metadata, and a shared lifetime record used by cached traversal handles. A host can therefore own non-interactive content, interactive controls, other hosts, or complete focus scopes.

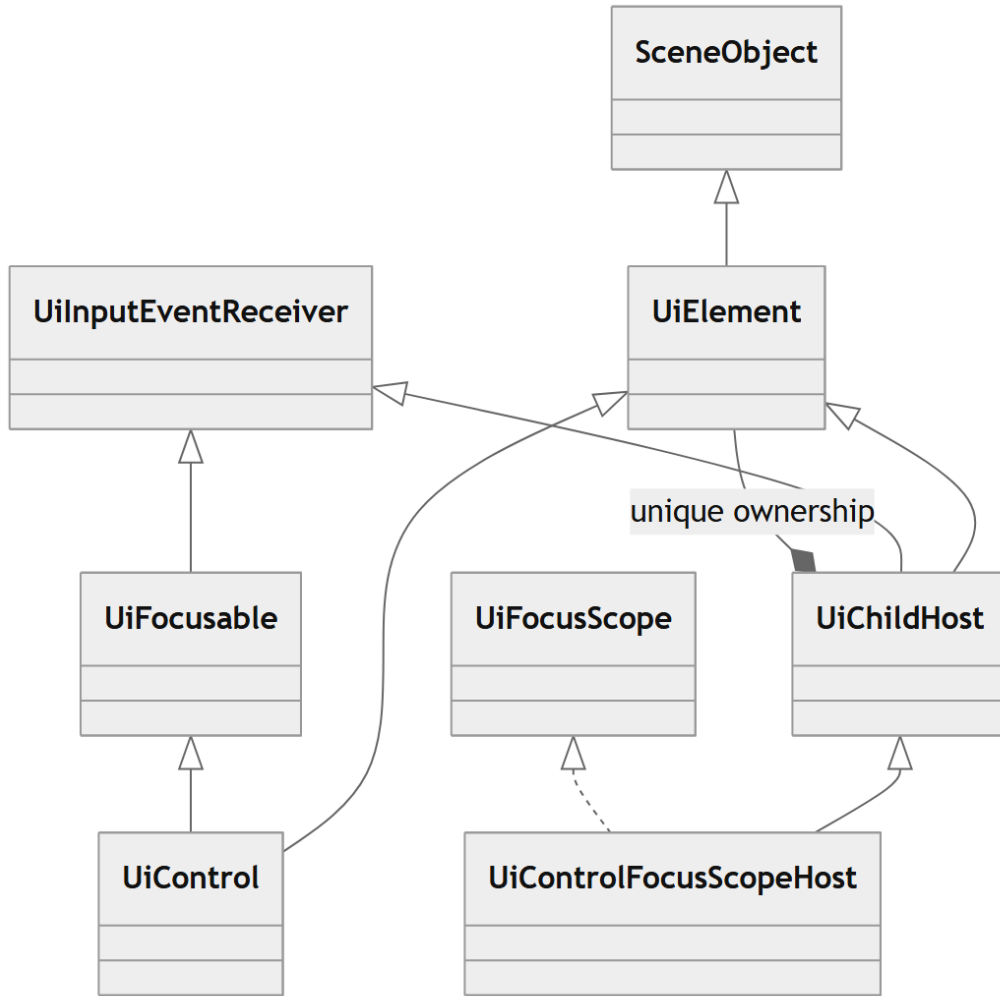


Figure 2. Core type roles and child ownership.

The important point is that **UiControl** and **UiChildHost** do not form one straight inheritance chain. A child-owning panel does not automatically receive atomic-control semantics. When a container also needs to manage a local control graph, it derives from **UiControlFocusScopeHost**, which combines **UiChildHost** and **UiFocusScope**.

3.3 One-frame data flow

The data flow is staged. Raw input becomes a UI frame snapshot and zero or more UI events. The window applies global priority. The chosen focus scope or subtree applies local rules. Updates and layout repair happen before render-command collection.

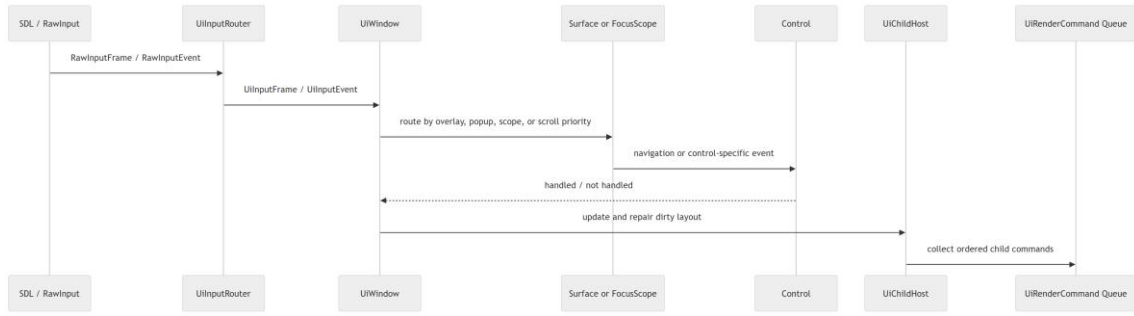


Figure 3. Simplified frame flow from raw input to rendering commands.

The framework treats a frame snapshot and a discrete event as different data forms. The snapshot is useful for held states and analog values. Events are useful for transitions such as press, release, pointer movement, text input, and wheel steps. This separation reduces the need for every widget to reconstruct edges from held state.

3.4 Boundary with the scene and renderer

UI objects are scene objects, but the UI tree has its own ownership inside a root element such as `UIWindow`. A scene creates the root window and keeps non-owning pointers to selected controls when needed. The window or another host remains the owner.

Rendering is also separated. UI nodes append typed `UiRenderCommand` values. A parent host can apply opacity, presentation translation, and clipping to the range produced by a child. The SDL render-command executor later performs the actual drawing. This means a button does not need to save and restore SDL renderer state itself for every parent effect.

4. UI Tree, Layout, Styling, and Lifecycle

4.1 Ownership and borrowed access

`UIClildHost::add_child` accepts `std::unique_ptr<UiElement>`. Successful insertion transfers exclusive ownership into a `ChildEntry`. `create_child<T>` performs construction and transfer in one step, then returns `T*` for borrowed access. `extract_child` performs the reverse operation and returns a `unique_ptr` to the caller. This interface makes transfer visible and supports transactional composite operations.

The choice is consistent with the C++ Core Guidelines: `unique_ptr` represents single ownership and a function parameter of that type expresses transfer [6]. A raw pointer returned from `create_child` does not own the object. Code that stores it must respect the lifetime of the host and child.

The framework still uses some borrowed registration pointers. `UIWindow` stores focus scopes, overlays, transient popups, tooltips, and a passive scroll target without taking separate ownership. Most registered objects are already owned somewhere in the window tree. Pruning code checks

whether they remain usable. This approach avoids shared-ownership cycles, but it is not formal lifetime safety. In particular, a `UiTooltip` owns its content but borrows its trigger. Window-managed removal can clear a trigger that leaves the same tree, yet arbitrary external trigger lifetimes still require caller discipline.

4.2 Safe traversal during mutation

UI callbacks can change the tree. A button callback may close a dialog, remove its parent, switch a tab, or rebuild a dropdown. Directly iterating a vector of `unique_ptr` values would be unsafe if the vector changes during a callback.

`UiChildHost` addresses this with `ChildLifetime` and `UiChildHandle`. A lifetime record stores an element pointer and generation. Cached logical and visual traversal lists keep a shared pointer to that record and the expected generation. When an entry is removed or replaced, the record is invalidated and its generation changes. A cached handle then resolves to `nullptr` instead of a deleted object. The host also separates logical order from visual order: logical order is used for update and ordinary tree behavior, while higher visual order receives input earlier and renders on top.

This mechanism reduces a practical class of use-after-free bugs. It does not make every borrowed pointer in the UI system automatically safe. Composite members and external scene pointers must still be updated at their lifecycle boundaries.

4.3 Layout invalidation and geometry

`screen_rect` is the common geometry result. A child's size change notifies its layout parent of intrinsic-layout invalidation. The dirty flag moves upward, and a host rebuilds layout on demand before input or rendering. This keeps geometry consistent without rebuilding every container unconditionally on every frame.

Common child options include anchors, alignment, desired size, and margins. `UiPanel` supports anchored placement. `UiListContainer` arranges children in one direction with spacing and cross-axis alignment. `UiGridContainer` uses rows and columns and can also build a two-dimensional focus relationship. `UiScrollContainer` owns one content surface and combines viewport clipping, scroll state, optional scrollbars, input routing, and a focus-scope proxy.

Layout geometry and presentation geometry are intentionally separate. A translation effect changes `presentation_translation`; hit testing uses the accumulated presentation rectangle, while the next layout pass still sees the original allocated rectangle. This avoids feedback where an animated offset changes the desired layout, which then changes the animation target again.

4.4 Render command composition

Each visible child appends commands to a shared vector. After a child returns, its parent transforms the command range as one unit. The parent may apply translation, multiply opacity, and intersect clip rectangles. This is a small command-composition layer between widgets and SDL.

The design gives parent containers control over subtree effects. A scroll viewport does not need every label and image to know about scrolling and clipping. A fading panel can apply alpha to all commands produced by its descendants. The renderer remains responsible for converting commands into SDL operations and restoring renderer state.

4.5 Style, theme, text, and effects

Styles are represented as base values plus optional field-level overrides. Interactive widgets resolve colors from enabled, hovered, pressed, focused, selected, and adjustment states. Visual-role enums allow the same widget class to request different semantic styles, such as a default button or a tab-like button.

`UiThemeManager` registers roots and reapplies a `UiTheme` through `UiThemeStyleResolver`. The built-in adapters are selected by exact dynamic type. Containers can mark internal children as composite implementation details so those children follow the outer component's semantic style rather than acting as unrelated theme roots. The current built-in theme is primarily color-based rather than a complete design-token or stylesheet system.

Text is separated into content and typography. `UiTextContent` can carry text or a localization key. Typography roles resolve to font size and related text rendering inputs. `UiLabel`, `UiTextBlock`, and `UiNumber` serve different display needs. `UiTextInput` keeps private editing texture state and accepts text-input and text-editing events in addition to navigation actions.

The effects layer contains small opacity and translation state machines used by image, label, and element variants. These effects change presentation without owning layout.

5. Input, Focus, Scrolling, and Window Arbitration

5.1 Input normalization

The engine input system first translates SDL data into `RawInputFrame` and `RawInputEvent` values. A scene gives raw input to gameplay receivers and also passes it through `UiInputRouter`. The router produces one `UiInputFrame` for held state and a list of `UiInputEvent` values for discrete changes. It can also synthesize repeated scrolling from the left gamepad stick.

`UiAction` contains device-independent intent: four navigation directions, Confirm, Cancel, Tab, Backspace, Delete, Home, End, Page Up, and Page Down. `UiInputEventType` keeps the payload

family: action press or release, mouse movement, pointer press or release, wheel, text input, text editing, or axis change. The event also keeps the source `InputDevice` and raw control. This makes it possible to share control logic without treating a mouse and gamepad as identical.

The current bindings are compile-time mappings in `ui_input_router.cpp`. Enter, Numpad Enter, and the gamepad south button map to Confirm. Escape and the gamepad east button map to Cancel. Arrow keys and the D-pad map to navigation. This is simple and deterministic, but it is not a user-rebindable UI action system. SDL controller mappings help normalize device layouts [2]; the engine then performs its own UI-specific mapping.

The gamepad scroll synthesizer treats the left-stick axes independently. After applying a dead zone, it accumulates both horizontal and vertical values and emits wheel-like steps. A diagonal stick can therefore scroll on both axes. The synthesizer caps steps per update and resets its accumulated state when the interaction session resets. This feature is useful because a scroll area can use one wheel-handling path for mouse and generated gamepad scroll events.

Unity 6.5's UI documentation also treats navigation events as intent that may come from a D-pad, joystick, Escape, Enter, or arrow keys rather than one specific device. It notes that a navigation event does not itself require focus to move [7]. Elysia uses a different implementation, but the same distinction supports the choice to separate normalized intent from the policy that consumes it.

5.2 Focus scopes as local navigation graphs

`UiFocusScope` is the boundary for a local focus domain. It exposes the element that defines the domain, whether the scope is active, its focused target, and whether it can navigate in a given direction. Two neighbor structures are used:

- `UiFocusNeighbors` links controls inside one scope;
- `UiFocusScopeNeighbors` links scopes at the window level.

This produces a two-level graph rather than one global list. A grid container can define left, right, up, and down relationships among its own cells. A window can define that a sidebar scope is left of a content scope without knowing every control inside either region.

`UiControlFocusScopeHost` stores a vector of focus entries. Each concrete container rebuilds this registry based on its structure. Navigation first gives the focused control a chance to consume the event. This matters for a slider or text field that uses an arrow for its own state. If the control does not consume the action, the host tries the registered neighbor. At a local boundary, `UiWindow` may move to a neighboring focus scope.

The graph is explicit. The current implementation does not search arbitrary screen geometry for the nearest control. Explicit neighbors give stable behavior for designed menus, but new containers must build the correct registry.

5.3 Focus repair and input-device policy

A stored focus target is valid only while the control is alive, active, visible, enabled, and still registered. `synchronize_focus_state` removes destroyed children, refreshes the registry, calls `ensure_valid_focus`, and applies the result to control visuals. If the current target becomes invalid, the host can restore its last usable target or focus the first available one.

The most recent input device changes the repair policy. Mouse input may leave a scope without a focused control when the pointer is not over a target. Keyboard and gamepad input instead restore the last usable target or choose the first available one. A control selected by the pointer can also become the preferred target for later restoration.

The W3C guidance describes predictable focus movement, visible focus, and a distinction between keyboard focus and selected state [3]. Elysia is not a web accessibility implementation, but those concepts help explain why one Boolean flag is not enough. The engine needs to know the active scope, the focused control inside it, the previous preferred target, and the device that caused the decision.

5.4 Delegated focus in nested containers

Composition creates nested scopes. A tab container owns a tab-bar scope and a page-view scope. A selected page may contain a chrome container, which may contain a scroll container, which may contain a list. If the outer composite copied all private focus logic from these containers, it would create inconsistent state.

`UiDelegatedFocusMixin` is a non-owning bridge. During focus-registry construction, it finds live controls in nested scope regions, adds them to the outer host's graph, and stores the actual owner scope for each control. When focus enters a delegated region, the mixin asks that region to focus a valid target. After local navigation, it synchronizes the nested result back to the outer host. The outer scope can therefore participate in window navigation while the inner scope remains responsible for its own focused target.

This design has a cost: composite containers must define transitions between regions and call synchronization at the correct lifecycle points. The benefit is that scroll, tab, and chrome regions remain separate focus domains instead of becoming one monolithic container class.

5.5 Scroll routing

`UiScrollContainer` combines a clipped viewport, one content payload, horizontal and vertical scroll state, optional scrollbars, and a content focus proxy. Mouse-wheel input first reaches suitable content; if content does not consume it, the scroll container updates its own offset.

Gamepad scrolling requires more policy. `UiWindow` collects focused scroll containers by visiting active and visible descendants before their parents. The first usable focused container receives the generated wheel event. If none is focused, the window first reuses the most recently pointer-promoted target and otherwise selects the first usable container found by the same descendant-before-parent traversal. Page Up and Page Down move by viewport-sized steps, while Home and End move to a boundary.

When a gamepad scroll actually moves content, a scope can temporarily clear a child focus visual and restore it on later navigation. This avoids showing a stationary focused row while the same stick scrolls the viewport. The current focused-gamepad route stops at the first usable container even when that container is already at its boundary. The implementation therefore does not support nested scroll chaining from an inner boundary to an outer container.

5.6 Window-level surface arbitration

`UiWindow` is a root UI container, not an operating-system window wrapper. It keeps window-level registrations and decides event priority. Figure 4 summarizes the current routing order.

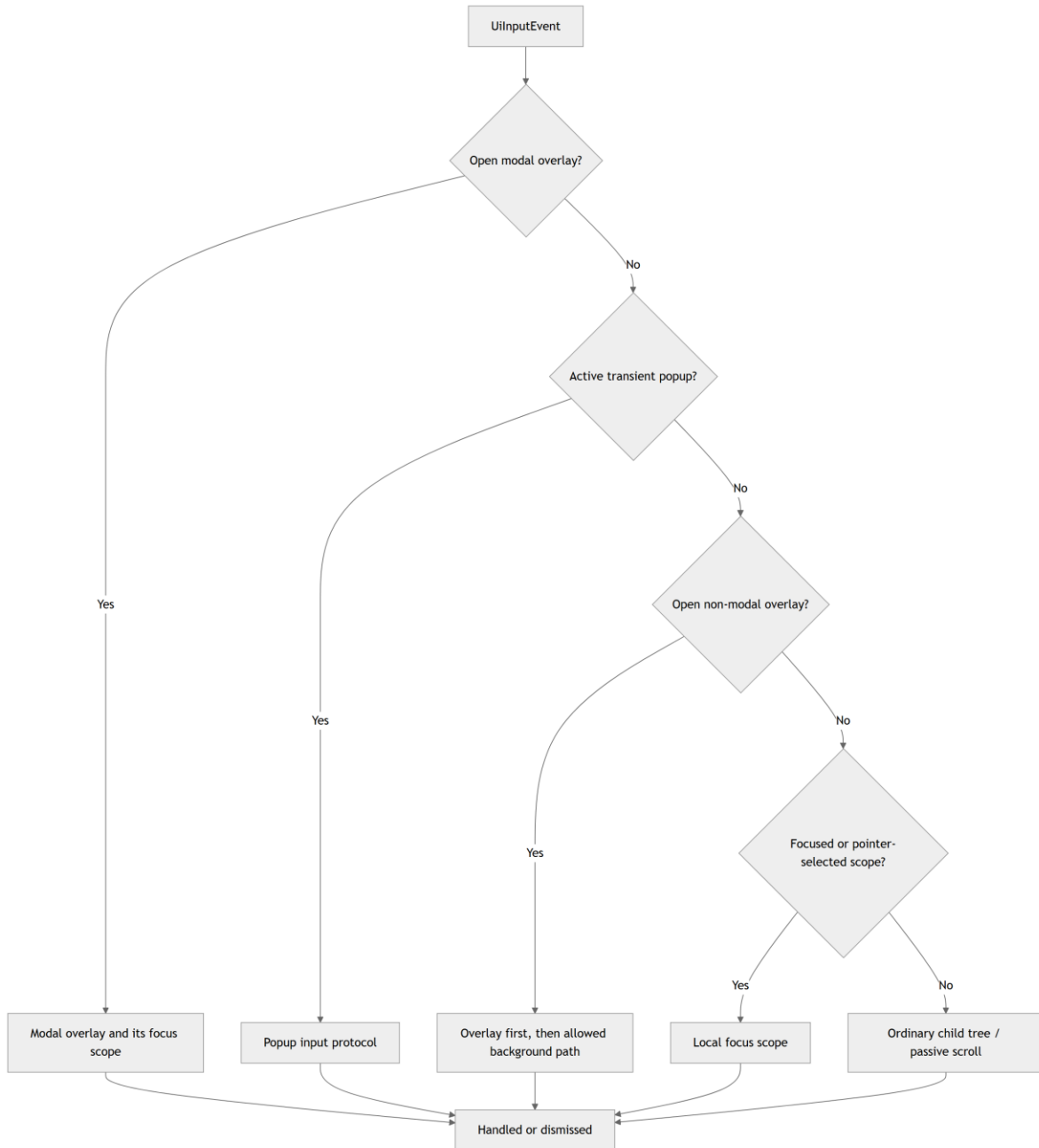


Figure 4. Simplified input priority in `UiWindow`.

An overlay is a direct child owned by the window and registered with `UiOverlayOptions`. Options describe whether it is open, modal, closable by Cancel, closable by an outside click, and where it should be placed. Opening an overlay can save the previous focus scope and focus the overlay. Closing it restores the previous scope only if that scope remains usable.

A transient popup uses a different protocol. The implementing component keeps ownership of its popup state and exposes input and rendering operations to the window. Only one transient popup is active. Activating another popup closes the previous one. This model fits a dropdown list that is visually outside ordinary child clipping but still belongs to the dropdown control.

A tooltip is passive. It does not implement focus or consume input. The window checks trigger reachability, ancestor visibility, clipping, presentation translation, modal state, popup occlusion, and pointer or focus reachability. Tooltip commands are submitted after ordinary children and the active transient popup. Epic's CommonUI design guidance makes a similar practical distinction: tooltips normally should not seize input, while popups and modal menus may need to block other UI handling [8].

The window also prunes registrations when objects leave its managed tree. This cleanup reduces stale pointers, but registration relationships are still borrowed. The API requires correct detach behavior, and not every possible external lifetime is represented by a checked handle.

6. Composite Component Case Study

6.1 Why `UiTabContainer` is the main case

`UiTabContainer` provides a compact case study that crosses several framework boundaries. It owns internal children, participates in layout, and delegates focus across nested regions. It also separates navigation from committed selection, coordinates callbacks, and repairs state after dynamic additions or removals. One clear invariant ties these responsibilities together: the number of tab labels must equal the number of pages.

Figure 5 shows the internal structure and the two index states.

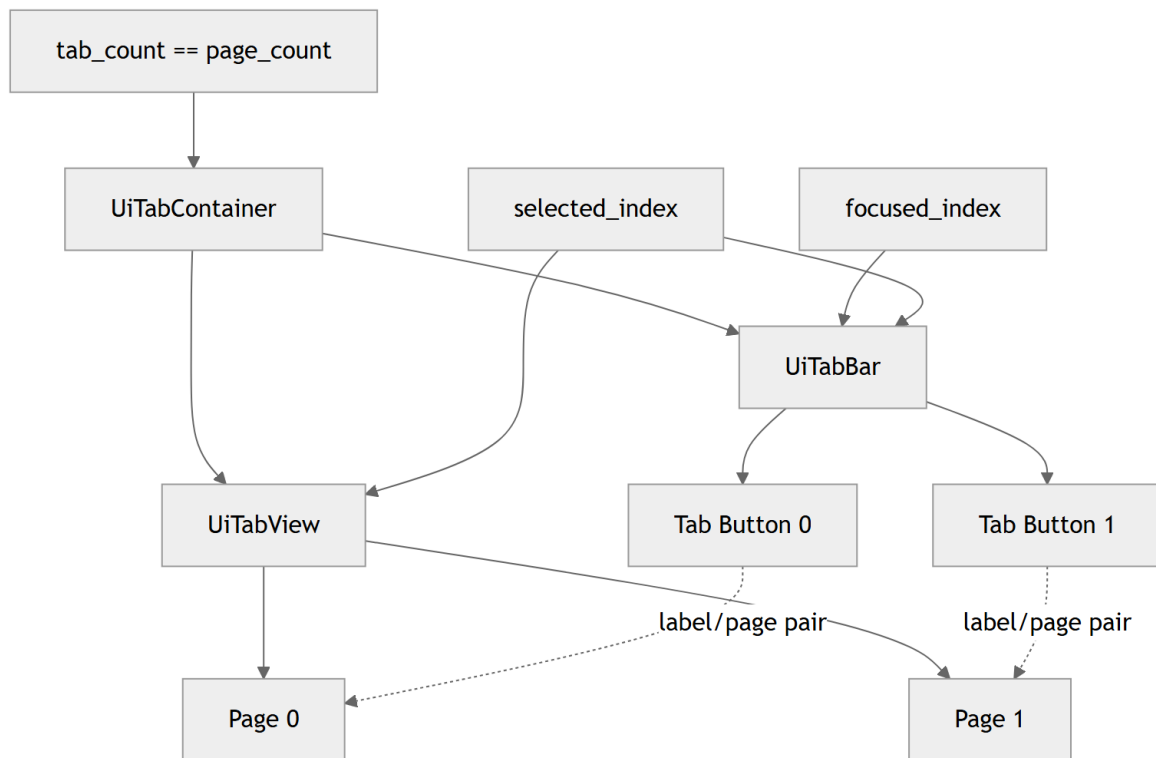


Figure 5. Composition and synchronized state in `UiTabContainer`.

6.2 Focus and selection are different states

The tab bar stores a selected button and a focused target separately. Left or right navigation changes focus. Confirm activates the focused button, which commits selection. The tab view displays only the selected page; other pages become inactive and invisible. This behavior matches the manual-activation option described by the W3C Tabs Pattern [4]. Elysia does not implement ARIA roles or browser tab order, but it uses the same useful interaction distinction.

Keeping two indices prevents navigation from causing an unwanted page switch. It also makes gamepad interaction predictable when a user is inspecting available tabs before confirming. Callers may set focus and selection through separate methods when a scene needs direct control.

6.3 Transactional addition and removal

`add_tab` receives a label and a `unique_ptr` page. It adds the page to `UiTabView` first, then asks `UiTabBar` to create the label button. If label creation fails, the method extracts the just-added page and returns it in `UiTabAddResult::rejected_page`. Ownership is not lost, and the pair count is restored before returning.

The first successful tab becomes selected in both the bar and view. Later additions leave the current selection intact. During internal changes, `_mutating` suppresses intermediate synchronization callbacks. After a successful change, the container marks layout dirty, refreshes focus entries, and checks the pair-count invariant.

Removal records the old focus and selection before extracting both the label and page. If the removed item was active, the same numeric index is used when a following item exists; otherwise the previous last item is used. Indices above the removed position shift down by one. External callbacks run after internal state becomes consistent.

The invariant is maintained through the dedicated tab API and checked with `assert(tab_count() == page_count())`. Inherited public child-manipulation functions can still bypass this path, so the guarantee depends on using the intended API.

6.4 Focus delegation between bar and page

The composite creates two delegated regions. The tab bar points downward to the tab view, while the tab view points upward to the bar. Down from the bar asks the selected page to focus its first available control. Up from page content restores the tab bar's current target. Other navigation stays inside the actual nested scope until it reaches a defined boundary.

This is an example of composition doing more than visual nesting. The outer component coordinates the regions, while each region keeps its own focus rules. The same mixin can support composites that contain different nested focus-scope types.

6.5 Supporting examples

`UiConfirmationDialog` demonstrates modal composition. It derives from `UiControlFocusScopeHost`, owns a `UiChromeContainer`, and builds title, message, Close, Cancel, and Confirm controls inside that structure. Registration defaults to a centered modal overlay. The action row places Cancel first, so the dialog's initial focus is the safer dismissal action. Confirm closes the overlay before calling the user callback. A callback may therefore switch scene or destroy the old UI without leaving the dialog open.

`UiTooltip` demonstrates mixed ownership. It owns displayed content through `unique_ptr` and borrows a trigger element. It is updated and rendered through `UiWindow` so it can appear above popup content without becoming a modal or focus scope. The window clears the trigger when it observes that a trigger has left its managed subtree, but the raw borrowed pointer is not a general lifetime handle. A caller that binds an external trigger must still clear or unregister it at the correct time.

7. Verification and Results

7.1 Verified environment

The implementation described in this paper was verified against Git revision `dfac1cee3d90e2476140701cd9c432a8e4e0ca36` on July 20, 2026, using a Windows MSVC Debug build. At that revision, the `engine/ui` module contained 133 C++ source and header files.

7.2 Automated verification

Tested concern	Examples of covered behavior
Layout and geometry	list extent, scroll measurement, subtree translation, transformed hit testing
Style and rendering	style cascade, theme propagation, interactive borders, renderer-state restoration
Focus lifecycle	empty scopes, nested propagation, preferred-leaf restoration, hidden or removed targets
Input and scrolling	tab focus versus selection, keyboard/gamepad matrices, two-axis synthesis, passive scroll targets
Layered surfaces	registration cleanup, modal and popup occlusion, clipping, tab-switch visibility
Dynamic tree safety	callback removal/reordering, cached traversal handles, exception propagation
Control state	group repair, callback preservation, editing textures, localized glyph reuse

Table 2. Automated behaviors covered by the UI test suite.

7.3 Interactive verification

The engine testbed includes a `UiTestScene` with a root `UiWindow`, a main `UiTabContainer`, nested tabs, scroll containers, selection controls, theme switching, a confirmation overlay, a dropdown popup,

and a tooltip. It provides an integration surface for behaviors that are difficult to capture through isolated assertions alone.

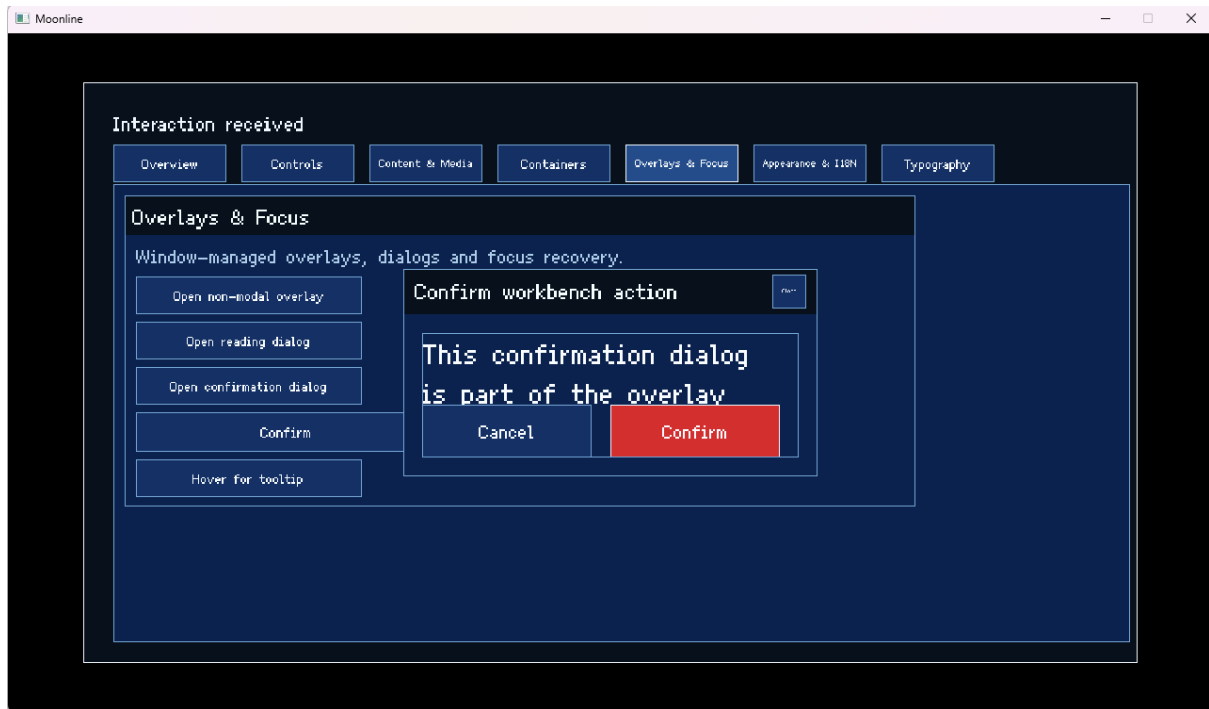


Figure 6. Interactive Elysia Engine UI test scene used for visual verification.

During a manual run of the rebuilt MSVC Debug binary, directional input moved focus across tab labels without changing the selected Overview page. Pointer input then activated the Containers layout page, opened the dropdown and confirmation overlay, and displayed the tooltip after its configured delay. Figure 6 shows the confirmation overlay during this integration run.

8. Limitations and Future Work

The current input system uses hard-coded mappings and does not yet support touch, gestures, or user rebinding. Focus navigation is based on explicit neighbor relationships, which works well for designed menus but does not provide general spatial navigation, and nested scroll containers do not currently chain input at their boundaries.

Presentation features are also limited in scope. The theme system is primarily color-based, text does not support rich-text composition, and effects are implemented as small state machines rather than a general animation timeline.

Verification has focused on the implemented behaviors under a Windows/MSVC configuration. Future work should expand cross-platform and real-device testing, measure larger UI trees, strengthen lifetime validation, and add accessibility-facing semantics.

9. Conclusion

Elysia Engine organizes its UI framework around a persistent tree with explicit separation between element state, interaction, ownership, local focus behavior, and window-level policy. This structure allows nested containers and composite controls to share common input, layout, and rendering mechanisms without collapsing into one monolithic widget hierarchy.

The main value of the design lies in the boundaries between these responsibilities. `unique_ptr` makes child ownership explicit, layout invalidation connects content changes to parent placement, and render-command composition lets parent containers apply clipping, opacity, and presentation translation across entire subtrees. `UiTabContainer` illustrates how these mechanisms can be combined while preserving separate focus and selection state.

The resulting framework provides a coherent base for menu and screen flows in Moonline and other Elysia Engine scenes. Its current scope remains narrower than a general-purpose GUI toolkit, but within that scope it demonstrates how ownership, focus, input routing, layered surfaces, and composition can be organized into a reusable and testable UI subsystem.

References

- [1] SDL Project, “SDL_Event,” *SDL2 Wiki*. [Online]. Available: https://wiki.libsdl.org/SDL2/SDL_Event. [Accessed: Jul. 20, 2026].
- [2] SDL Project, “SDL_GameControllerAddMapping,” *SDL2 Wiki*. [Online]. Available: https://wiki.libsdl.org/SDL2/SDL_GameControllerAddMapping. [Accessed: Jul. 20, 2026].
- [3] W3C WAI-ARIA Authoring Practices Task Force, “Developing a Keyboard Interface,” *ARIA Authoring Practices Guide*. [Online]. Available: <https://www.w3.org/WAI/ARIA/apg/practices/keyboard-interface/>. [Accessed: Jul. 20, 2026].
- [4] W3C WAI-ARIA Authoring Practices Task Force, “Tabs Pattern,” *ARIA Authoring Practices Guide*. [Online]. Available: <https://www.w3.org/WAI/ARIA/apg/patterns/tabs/>. [Accessed: Jul. 20, 2026].
- [5] O. Cornut and Dear ImGui contributors, “FAQ (Frequently Asked Questions),” *Dear ImGui Documentation*. [Online]. Available: <https://github.com/ocornut/imgui/blob/master/docs/FAQ.md>. [Accessed: Jul. 20, 2026].
- [6] B. Stroustrup and H. Sutter, eds., “C++ Core Guidelines,” Jun. 14, 2026. [Online]. Available: <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines>. [Accessed: Jul. 20, 2026].
- [7] Unity Technologies, “Navigation events,” *Unity 6.5 Manual*. [Online]. Available: <https://docs.unity3d.com/Manual/UIE-Navigation-Events.html>. [Accessed: Jul. 20, 2026].
- [8] Epic Games, Inc., “Design Guidelines for Using CommonUI in Unreal Engine,” *Unreal Engine 5.8 Documentation*. [Online]. Available: <https://dev.epicgames.com/documentation/en-us/unreal-engine/design-guidelines-for-using-commonui-in-unreal-engine>. [Accessed: Jul. 20, 2026].